

The C4 Component — Explained Simply

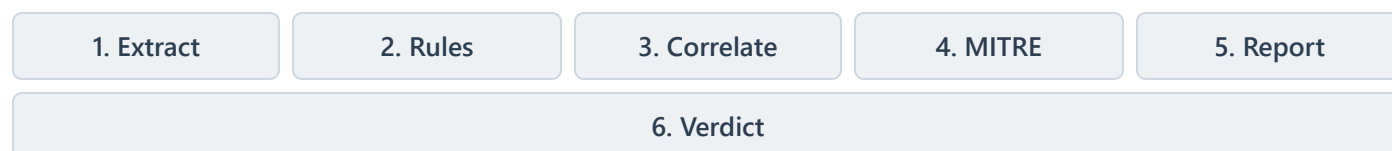
A file-by-file, function-by-function plain-English walkthrough of your Browser Artifact Forensics component (WebSentinel — R26-CS-003)

What C4 does, in one sentence: it reads a Chrome browser's saved files straight off the disk (history, cookies, passwords, downloads, extensions, open tabs), figures out which of that activity looks like a hack, connects the suspicious pieces together into a story, labels that story using the industry-standard MITRE ATT&CK framework, and hands out a final risk verdict — all without ever changing the original evidence.

Files covered in this document (in the order data flows through them):

1. `extractor.py` — Stage 1: pulls raw evidence off disk
2. `crypto.py` — Stage 1b: decrypts saved passwords
3. `rules.py` — Stage 2: flags obviously suspicious single events
4. `correlation.py` — Stage 3: connects evidence across different artifact types (7 detectors)
5. `mitre.py` — Stage 4: labels findings with official attack technique names
6. `reporter.py` — Stage 5: turns results into HTML / JSON / SIEM reports
7. `service.py` — Stage 6: the "front door" that runs everything and gives the final verdict
8. `__init__.py` — the small file that decides what the rest of WebSentinel is allowed to call

The pipeline, at a glance



Every browser artifact event travels through these six stages, left to right. Each stage adds more information onto the event (or onto the case as a whole) without ever touching the original browser files.

This is the biggest file, and its job is simple to state: **go get the evidence off the disk, safely, and turn it into a clean, uniform list of "events."** Every other file in C4 only ever works with that event list — none of them touch the raw browser files directly.

Analogy: think of this file as the crime-scene technician. It doesn't decide what's suspicious — it just carefully photographs and bags every piece of evidence (history entries, cookies, downloads, saved passwords, extensions, open tabs) without disturbing the original scene.

Small helper functions first

Function	In simple words
<code>chrome_time(ts)</code>	Chrome stores dates as a huge number of microseconds since the year 1601 (not a normal date). This function converts that odd number into a normal, readable date and time.
<code>sha256(path)</code>	Computes a unique "fingerprint" of a file. If even one byte of the file changes later, this fingerprint changes completely — used to prove evidence wasn't tampered with.
<code>get_chrome_path()</code>	Figures out where the Chrome profile actually lives on this machine (checks a few standard locations, including WebSentinel's own automated browser profile).
<code>find_file(profile, *names)</code>	Looks for a file under a few possible names/locations and returns the first one that exists.
<code>safe_copy(src, dst_dir, name)</code>	Copies a database file to a safe working folder before reading it. If Chrome has the file locked (because the browser is open), it falls back to a special read-only SQLite "backup" trick that can still read a locked file without disturbing it.
<code>query(db, sql)</code>	A small wrapper that runs one SQL query against a SQLite database file and returns the rows, or an empty list if anything goes wrong.
<code>_event(...)</code>	Builds one standard "event" dictionary — the common shape every single piece of evidence gets turned into, no matter which artifact type it came from. This uniform shape is what lets every later stage treat all evidence the same way.

The eight extraction functions — one per artifact type

Function	What it reads	Simple explanation
----------	---------------	--------------------

<code>extract_history()</code>	<code>History</code> file	Reads every URL the user visited and when, straight from Chrome's own history database.
<code>extract_cookies()</code>	<code>Network/Cookies</code> file	Reads saved cookies. If the file is locked because the browser is currently open, it falls back to reading the cookies live from the running browser session instead (via <code>cookies_from_live()</code>), and clearly marks those as "live" so a report never pretends they came from disk.
<code>extract_downloads()</code>	<code>History</code> file (downloads table)	Reads every downloaded file's name, source, and size. If the file type looks dangerous (like <code>.exe</code>) or Chrome itself flagged it, it's marked risky, and if the file still exists on disk its SHA-256 fingerprint is computed for real.
<code>extract_credentials()</code>	<code>Login Data</code> file	Reads every saved username/password record. The password itself is encrypted, so this function calls into <code>crypto.py</code> to decrypt it before recording the result (masked, never in full plaintext).
<code>extract_extensions()</code>	<code>Extensions/</code> folder + <code>Secure Preferences</code> file	Finds every installed browser extension from both places Chrome can store them, checks its requested permissions against a list of risky ones (like reading all URLs, or intercepting web requests), and flags extensions that weren't installed from the official Chrome Web Store ("side-loaded").
<code>extract_clusters()</code>	<code>History</code> file (Chrome 110+ tables)	Reads Chrome's own built-in "browsing journeys" grouping feature, when available, as bonus context.
<code>extract_sessions()</code>	<code>Sessions/</code> folder (SNSS files)	Recovers which browser tabs were open — even tabs whose URLs were never visited according to History (e.g. history was cleared but the tab stayed open). These are raw binary log files, so a pattern-matching scan pulls out URL text from them directly.
<code>extract_local_storage()</code>	<code>Local Storage/leveldb</code> folder	Scans the browser's local-storage database to see which websites ("origins") have stored data on this machine, even ones with no matching history entry.

Putting it together

Function	In simple words
<code>collect_manifest(profile)</code>	Builds a "chain of custody" record — for every source database file, its fingerprint (hash), size, and last-modified time. This is what proves later on that the analysis never altered the original evidence.
<code>run_extraction(...)</code>	The orchestrator. Calls all eight extraction functions above, combines every event into one big list, builds the manifest, and returns everything as a single result dictionary. This is the one function the rest of C4 actually calls to kick off Stage 1.

2. crypto.py

Stage 1b — Password decryption · 137 lines

Chrome never stores saved passwords as plain text — they're always encrypted. This small, focused file exists purely to reverse that encryption so C4 can prove a saved password is real (without ever needing to expose it in full).

Analogy: *it's like a locksmith who can open a specific type of safe (Chrome's password vault) using the correct combination (the Windows-protected master key) — but instead of handing you the jewels inside, they only confirm "yes, there's something real in here" and show you a partially covered glimpse.*

How Chrome's password encryption actually works (from the file's own comments)

1. Chrome keeps one AES encryption key, and that key itself is protected by Windows' own built-in data-protection system (called **DPAPI**), stored in a file called `Local State`.
2. Because DPAPI is tied to the specific Windows user account, only that same user's account can unlock the key — which matches exactly how C4 is meant to run (as the same user who owns the browser).
3. Each saved password is encrypted individually using that key with AES-256-GCM (a strong, modern encryption method).

Function	In simple words
<code>_dpapi_decrypt(blob)</code>	Talks directly to a core Windows security function (<code>CryptUnprotectData</code>) to unlock something that Windows itself encrypted for the current-user. Used both to unlock Chrome's master key and for old-style Chrome passwords.

<code>load_master_key(profile_path)</code>	Finds Chrome's <code>Local State</code> file, pulls out the encrypted master key stored inside it, and uses <code>_dpapi_decrypt</code> to unlock it — giving back the real AES key.
<code>_mask(plaintext)</code>	Turns a real decrypted password into a safe preview, e.g. <code>"hunter2"</code> becomes <code>"h*****2"</code> . Proves the decryption genuinely worked without ever writing the real password into a report file.
<code>decrypt_password(blob, master_key)</code>	Decrypts one saved password. Handles both the modern format (AES-256-GCM) and the old legacy format (plain DPAPI). Always returns a status (success / no-key / failed / etc.) plus the masked password and its true length.

Important safety detail: a flag called `REVEAL_PLAINTEXT` is set to `False` by default. This guarantees that no forensic report or file C4 writes to disk ever contains a full, real password — only masked previews and a "decryption succeeded" status.

3. rules.py

Stage 2 — Single-artifact rules (R01–R06) · 114 lines

Once Stage 1 has produced a clean list of events, Stage 2 looks at each event **by itself** (not yet comparing it to other events) and asks: "is this one thing obviously suspicious?" There are six fixed rules.

Rule	Function	What it looks for, simply
R01	<code>rule_suspicious_domain</code>	Was a history visit to a domain on a known-risky list (e.g. <code>pastebin.com</code> , <code>ngrok.io</code> , URL shorteners)?
R02 / R02b	<code>rule_dangerous_download</code>	Was a downloaded file's extension dangerous (e.g. <code>.exe</code> , <code>.bat</code> , <code>.ps1</code>), or did Chrome's own built-in safety check flag it?
R03	<code>rule_sensitive_cookie</code>	Is a cookie's name one that typically holds a login/session token (e.g. <code>"session"</code> , <code>"auth"</code> , <code>"jwt"</code>)?
R04	<code>rule_credential_record</code>	Every saved password record is flagged — a credential that's never been used yet scores even higher, since it looks like it may have been freshly planted/stolen rather than genuinely used by the owner.
R05	<code>rule_risky_extension</code>	Does an installed extension ask for dangerous permissions (e.g. <code>read all websites</code> , <code>read cookies</code> , <code>use the debugger</code>)?

R06	<code>rule_url_burst</code>	A "batch" rule that looks at the whole history list at once: were 20+ URLs visited within any 60-second window? That's a speed no human clicking normally achieves — a sign of a bot or malware script.
-----	-----------------------------	---

The orchestrator

`apply_single_artifact_rules(events)` runs every rule above against every event, collects whichever rules fired on it, adds up their scores into that event's `anomaly_score`, and marks the event as flagged if any rule matched. It then separately runs the R06 "burst" rule across the whole list.

Why keep this separate from Stage 3 (correlation)? Rules here only ever look at **one** event in isolation — fast, simple, cheap checks. Stage 3 is reserved for smarter checks that need to compare *multiple different* pieces of evidence against each other, which is a fundamentally different (and more expensive) kind of analysis.

4. correlation.py

Stage 3 — Cross-artifact correlation · 464 lines

This is the most advanced file in the component. Instead of judging one event at a time like Stage 2, it looks for **patterns across different types of evidence** — the kind of connections a human analyst would draw by comparing several clues together. It runs seven independent detectors, each looking for a different shape of suspicious pattern, all within a rolling 2-minute time window (`WINDOW_SECONDS = 120`).

Analogy: Stage 2 is like a security guard checking each visitor's ID card one by one. Stage 3 is the detective who notices "this same visitor's ID showed up at three different doors within two minutes, right after a suspicious package was dropped off" — a pattern no single ID check alone could reveal.

Detector A — Co-occurrence (`run_cooccurrence`)

For every domain, counts how many *different types* of evidence (history, cookie, download, credential, etc.) show up together within a 2-minute window. The more different types clustered on one domain, the more suspicious — a domain with just a history visit is normal, but a domain with history + cookie + download + credential all at once looks like an active compromise.

Detector B — Orphan detection (`run_orphan_detection`)

First builds the full list of domains the user's history says they actually visited. Then checks: does every cookie / credential / download / extension / session / local-storage entry belong to a domain that's actually in that visited list? If an artifact exists for a domain with **no** matching history visit, it's an "orphan" — a sign of injection, or that history was deliberately wiped while other traces were left behind.

Detector C — Temporal anomaly (`run_temporal_anomaly`)

Builds a personal "normal hours" profile for this specific user, purely from their own history (e.g. this user is normally active 9 AM–5 PM). Any hour with less than 2% of their total activity counts as "personally inactive." Then it flags any risky-type event (download, credential, cookie, etc.) that happens during one of those personally-unusual hours — catching things like 3 AM activity that doesn't match how this particular person normally browses.

Detector D — Attack chain detection (`run_attack_chain_detection`)

Looks for a specific **ordered** sequence of event types on the same domain within the time window — e.g. browse → download → credential access, in that exact order. This is stricter than Detector A because order matters here, not just "these things happened near each other." A handful of known dangerous sequences (`CHAIN_PATTERNS`) are checked, each with its own risk score.

Detector E — Domain risk clustering (`run_domain_risk_clustering`)

Scores every domain overall by combining: how many different artifact types touched it, how many of its events were already flagged, and how many events it has in total. Domains that score high (and touch 2+ artifact types) become "risk clusters" — effectively a ranked list of "which domains deserve the most attention."

Detector F — Credential reuse (`run_credential_reuse`)

Groups saved credentials by username and checks: does the same username appear saved against two or more *different* domains? If one of those sites gets breached, the same password can unlock the others too — this detector catches that "blast radius" risk, which is invisible if you only look at one login record at a time.

Detector G — Download → exfiltration (`run_download_exfil`)

For every download, checks whether the browser navigated to a *different* domain within the next 2 minutes. This is the classic "drop a tool, then call home / upload data" pattern seen in real intrusions — a file lands on disk, and shortly after, outbound traffic goes somewhere else.

Orchestrator

`run_correlation(events)` runs all seven detectors above, collects every finding, and also returns a `summary` dictionary with a count from each detector — this is what later confirms "all seven detectors fired" in your test.

5. mitre.py

Stage 4 — MITRE ATT&CK mapping · 265 lines

MITRE ATT&CK is a globally recognized catalogue of real attacker techniques, used by security teams everywhere. Instead of a report just saying "this looks bad," this file translates every finding from Stage 2 and Stage 3 into an official technique name, ID, and category — so any analyst reading the report instantly understands what class of attack it resembles.

MITRE_MAP — the lookup table

A big dictionary that maps specific patterns (e.g. `"download+credential"` , or rule `"R01"`) to a MITRE technique — its ID (like `T1555.003`), its human-readable name (e.g. "Credentials from Web Browsers"), its broad tactic category (e.g. "Credential Access"), a default severity, and a description.

The mapping functions

Function	Maps findings from
<code>map_cooccurrence</code>	Detector A findings
<code>map_orphan</code>	Detector B findings
<code>map_temporal</code>	Detector C findings
<code>map_attack_chain</code>	Detector D findings
<code>map_domain_cluster</code>	Detector E findings
<code>map_credential_reuse</code>	Detector F findings
<code>map_download_exfil</code>	Detector G findings
<code>map_rule_flags</code>	The R01–R06 rule flags from Stage 2

Each of these takes a raw finding and attaches its matching MITRE technique info to it. If a pattern isn't in the lookup table, a sensible fallback technique is used instead, calculated from the finding's own score via `_severity(score)` (≥ 70 = High, ≥ 40 = Medium, else Low).

Orchestrator

`run_mitre_mapping(correlation_result, events)` runs every mapping function above, merges all the labeled findings into one list (`all_findings`), counts how many are High/Medium/Low severity, and sorts everything with the most severe findings first.

6. reporter.py

Stage 5 — Reports · 237 lines

This file takes the fully analyzed result and turns it into formats a human or another system can actually use.

Function	Produces	In simple words
<code>generate_html_report(result)</code>	A complete HTML web page	Builds a styled, self-contained report page with an executive summary, the evidence manifest (with hashes), a MITRE findings table, and a timeline of the top 50 flagged events — the kind of document you'd hand to a stakeholder or examiner.
<code>generate_siem_export(result)</code>	A JSON structure	Converts every MITRE finding and every high-risk event into a standard record format compatible with real Security Operations Center tools (like Splunk, QRadar, or ELK) — so a real organization could feed C4's output straight into their existing monitoring systems.
<code>save_all_outputs(result, output_dir)</code>	Three files on disk	Writes the full JSON result, the HTML report, and the SIEM export to disk with timestamped filenames, and returns their paths.

7. service.py

Stage 6 — Orchestration & verdict · 146 lines

This is the "front door" of the whole component. The comment at the top explains why it exists: the original C4 code was written as a standalone tool, and this file adapts it into something the rest of the WebSentinel project (`core/main.py`) can simply call as a function.

Function	In simple words
<code>get_default_profile_path()</code>	Returns where C4 will look for a Chrome profile if nothing else is specified.

<code>run_forensic_analysis(...)</code>	The master orchestrator — runs Stage 1 (<code>run_extraction</code>) → Stage 2 (<code>apply_single_artifact_rules</code>) → Stage 3 (<code>run_correlation</code>) → Stage 4 (<code>run_mitre_mapping</code>), bundles everything into one result dictionary, optionally saves the reports (Stage 5), and remembers the result for later. This is the single function call that runs the entire pipeline end to end.
<code>get_summary(result)</code>	Builds the final compact verdict. It counts High/Medium/Low severity findings, combines them into one weighted <code>risk_score</code> (0–100, with diminishing returns for extra findings so 20 similar findings don't unfairly outscore 3 severe ones), and converts that score into a plain verdict: CLEAN → LOW → MEDIUM → HIGH → CRITICAL .
<code>get_last_result()</code>	Returns whatever the most recent analysis produced, without re-running anything.
<code>render_last_html()</code> / <code>render_last_json()</code> / <code>render_last_siem()</code>	Convenience functions that render the last saved result into each output format on demand (used by the dashboard).
<code>report_filename(kind)</code>	Builds a timestamped filename for a given report type.

How the risk score is actually calculated (from `get_summary`): High findings are worth 100 points, Medium 55, Low 20. These are averaged across all findings, then scaled by how many findings there are in total (more findings push the score higher, but with diminishing returns past about 10 findings) — so one lucky High finding on an otherwise quiet profile doesn't automatically scream "CRITICAL," but a case with many severe findings correctly does.

8. `__init__.py`

The public interface · 23 lines

The shortest file, but an important one. It decides exactly which functions from `service.py` are allowed to be imported by the rest of the project (like `core/main.py`) — things like `run_forensic_analysis` and `get_summary`. Everything else inside the C4 folder (all the internal extraction, rules, correlation, and MITRE logic) stays private implementation detail that outside code never needs to know about directly.

Analogy: if the whole C4 folder were a shop, `__init__.py` is the shop's front counter — it's the only part a customer (the rest of WebSentinel) is meant to interact with, while all the actual

work happens in the back rooms (`extractor.py` , `correlation.py` , etc.).

Putting it all together — the full journey of one event

To make the whole pipeline concrete, here's what happens to a single malicious download, start to finish:

1. **extractor.py** reads the download row out of the copied `History` database, notices its `.exe` extension, computes its real file hash, and creates one standard "event" for it.
2. **rules.py** (Stage 2) sees the `.exe` extension and fires **R02**, marking the event as individually risky.
3. **correlation.py** (Stage 3) notices this download's domain also has a credential and a history visit nearby in time (Detector A), that it's followed by navigation to a different domain (Detector G), and that it fits the browse→download→credential order (Detector D) — three separate detectors all pick it up from different angles.
4. **mitre.py** (Stage 4) labels these findings with real technique IDs like `T1204.002` (Malicious File) and `T1567` (Exfiltration Over Web Service).
5. **reporter.py** (Stage 5) writes this into the HTML timeline, the MITRE findings table, and the SIEM export.
6. **service.py** (Stage 6) folds its severity into the overall risk score, which pushes the final verdict toward HIGH or CRITICAL.

Big picture takeaway: each file has exactly one job, and data flows through them in a strict one-way pipeline — raw disk bytes go in at `extractor.py` , and a human-readable risk verdict comes out at `service.py` . No file ever modifies the original browser evidence; everything works on copies, and every source file is fingerprinted so that can be proven.